

BorderPulse — Project Audit & Hackathon Readiness Report

False-Alarm-Resilient Border Surveillance Sentinel System — full codebase review, problem-statement alignment analysis, and improvement roadmap

Prepared for: Thoufikur (SVCE) — for a friend's hackathon project at D:\hackelite

Reviewed by: Claude, based on a full read of the project source code, documentation, and three existing engineering-spec PDFs found in the repository

Problem Statement (as given):

“False-Alarm-Resilient Border Surveillance Sentinel System — Automated surveillance systems deployed in natural terrain frequently generate excessive false alarms due to environmental factors such as wind, vegetation, or wildlife movement, undermining operator trust in the system. Develop a hardware system capable of reliably distinguishing genuine human/vehicle intrusion from routine environmental movement in an outdoor setting.”

What This Report Covers

- 1. The problem space — how existing systems (in India and globally) solve false-alarm-resilient perimeter surveillance
- 2. What BorderPulse actually is — architecture, tech stack, current build status
- 3. Alignment check — does the project actually solve the stated problem, feature by feature
- 4. **The single most important bug found** — a logic gap that works against the hackathon's core requirement
- 5. File-by-file map of the codebase, and exactly how to run it
- 6. What's broken, unfinished, or only simulated right now
- 7. UI/UX assessment — what to restyle without touching structure
- 8. New feature recommendations, prioritized
- 9. YOLO model, detection classes, and sensor threshold reference (incl. why 'rain/disaster detection via YOLO' isn't the right approach)
- 10. The Antigravity fix prompt (companion file: ANTIGRAVITY_FALSE_ALARM_FIX_PROMPT.md)

1. Existing Solutions — India and Worldwide

1.1 India: government-scale multi-sensor fusion

India's border security agencies (BSF) run the **Comprehensive Integrated Border Management System (CIBMS)**, deployed in pilot stretches on the Indo-Pakistan and Indo-Bangladesh borders, with a flagship riverine deployment at Dhubri, Assam called **BOLD-QIT** (Border Electronically Dominated QRT Interception Technique). The architecture has three tiers: (1) sensors — thermal imagers, infrared/laser intrusion beams, aerostats, unattended ground sensors, radar, sonar for riverine stretches, and fibre-optic sensing; (2) a wired/wireless communication backbone; (3) a command-and-control centre that fuses all sensor feeds, **correlates alerts to suppress false alarms**, and hands off to a Quick Reaction Team.

The core lesson from CIBMS that is directly relevant here: **no single sensor is trusted alone**. False-alarm resilience comes from cross-checking a vision/IR trigger against an independent physical sensor (ground vibration, radar, acoustic fibre) before an alert is escalated to a human response team.

1.2 Global commercial and research approaches

- **AI video analytics (Lumana, Mobotix, Guardian, etc.):** classify every motion event as person / vehicle / animal / environmental-noise (wind-blown foliage, rain, insects near the lens) before deciding whether to alert. Some adapt sensitivity in real time based on wind conditions — filtering tightens on windy days and relaxes when wind subsides. Industry figures cited by these vendors: 90–99% of raw motion triggers in outdoor CCTV are false, and AI classification cuts false alarms by up to 90%.
- **Distributed Acoustic Sensing / fibre-optic perimeter sensors (AP Sensing, FEBUS):** turn a buried optical fibre into thousands of virtual microphones along a fence line, classifying the vibration signature (footstep vs. digging vs. driving vs. wind/rain) with ML — reliable rejection of weather and wildlife by signal signature, not by camera image.
- **EU FOLDOUT research project (forested/green border surveillance):** a published multi-sensor fusion study found that combining passive infrared detectors with automatic person detection from thermal and visual cameras, using a rule engine over weighted evidence maps, eliminated many single-sensor false detections and improved accuracy by up to 50% versus a single sensor such as radar alone in dense vegetation — radar alone is explicitly called out as prone to false triggers from branches/leaves moving in wind or rain.

- **Common design pattern across all of the above:** weighted/fused evidence from at least two independent modalities, temporal confirmation over multiple readings/frames rather than a single trigger, and explicit classification (not just motion detection) before any alarm is raised.

1.3 What existing solutions still struggle with

- Government-grade systems (CIBMS, BOLD-QIT) are expensive, purpose-built for fixed border stretches, and not something a low-cost prototype can replicate hardware-for-hardware — but the fusion + classification + temporal-confirmation *principle* is fully replicable at hobbyist/prototype scale, which is exactly what a webcam + YOLO + ESP32 + simulated-sensor architecture is trying to approximate.
- Even mature commercial AI camera systems still report the 90–99% false-alarm baseline for raw motion sensors — the AI layer is what brings that down, not the camera itself. This means the classification and fusion logic (the software) is the actual differentiator for a hackathon judge, not the camera hardware.
- Wind-adaptive sensitivity (dynamically raising the confirmation bar in high wind) is called out repeatedly in the literature as a distinguishing feature — most basic student-level intrusion detection prototypes skip this entirely and just run YOLO with a fixed confidence threshold.

2. What BorderPulse Actually Is

BorderPulse is a genuinely substantial, mostly-working prototype — not a stub. It is not a generic "intrusion detector"; it is specifically architected around the false-alarm problem: it classifies objects by type (not just "motion"), requires either high-confidence vision or multi-frame temporal confirmation plus optional sensor corroboration, and keeps a documented event lifecycle. This is the right shape of solution for the problem statement.

Pipeline: Camera (thread) → YOLO11n detection + ByteTrack → polygon restricted-zone check (ray-casting, foot-point) → sensor fusion (vision + radar + ground + temporal, weighted) → decision engine (state machine, per track/zone) → event manager (cooldown/dedup) → evidence capture (pre/post buffer, snapshot + clip) → ESP32 buzzer + Supabase (Postgres + Storage) + React dashboard (WebSocket live stream)

Layer	Technology	Status
Detection	YOLO11n (Ultralytics) + ByteTrack	Working — loads and infers on laptop webcam
Zone engine	Ray-casting point-in-polygon, normalized 0–1 coords	Working — 13/13 unit tests pass
Decision engine	Per (track_id, zone_id) state machine, 10 states	Working, but see Section 4 (critical bug)
Sensor fusion	Weighted vision/radar/ground/temporal score	Implemented correctly, but bypassed for 'person' class
Radar	RCWL-0516, planned	Not wired — simulated only, and hardcoded 'OFFLINE' label in the live stream regardless of simulation toggle
Ground sensor	Vibration sensor via ESP32 GPIO	Simulated by default; code path exists for real ESP32 reading
ESP32	Wi-Fi HTTP REST, buzzer + future sensors	Firmware written, not flashed; board variant unconfirmed

Layer	Technology	Status
Backend	FastAPI + WebSocket	Running, 13/13 API endpoints return 200
Database	Supabase Postgres, 9 tables, RLS enabled	Connected and writing
Storage	Supabase Storage — event-images, event-videos	Buckets exist; end-to-end upload not yet verified live
Frontend	React + Vite + Tailwind, 10 pages	All 10 pages implemented; reviewed 4 of them directly, functional and coherent — see Section 6

3. Alignment Check — Does This Project Actually Solve the Problem Statement?

Short answer: the **architecture** is well-aligned — it's built around classification and fusion, which is the correct approach per Section 1. But there is one implementation bug that actively works against the stated goal, plus a few gaps versus what "state of the art" false-alarm resilience looks like. Below is a feature-by-feature check against the problem statement's own wording.

Problem statement requirement	Implemented?	Notes
Distinguish genuine human/vehicle intrusion from routine movement	Partially	YOLO only fires on trained COCO classes (person, vehicle, several animals/birds) — this already filters out raw pixel-level foliage flicker, which a naive motion-detector approach would not. Good baseline.
Resilience to wind / vegetation movement specifically	Not explicit	There is no wind/weather input anywhere in the system. Resilience is <i>implicit</i> (YOLO doesn't classify leaves as objects) rather than <i>adaptive</i> (raising the confirmation bar when wind is high, as commercial systems in Section 1.2 do). This is the clearest feature gap vs. the problem statement's specific wording.
Resilience to wildlife movement	Yes, by design	Animal/bird classes are detected, logged, and explicitly excluded from triggering the buzzer alarm by default (config: only 'person' escalates to critical). This is correct and matches the problem statement well.
Reduce operator alert fatigue / false alarms	Undermined by a bug	See Section 4 below — the current code makes this worse for the 'person' class specifically, the one class it should be most careful about.
Hardware system (not just software)	Partially	ESP32 + buzzer are real hardware, but not yet flashed/wired. Radar and ground sensor are still simulated. For a hackathon demo this needs at minimum the ESP32 buzzer path working end-to-end.

Problem statement requirement	Implemented?	Notes
Multi-sensor evidence fusion	Implemented	FusionEngine (vision/radar/ground/temporal weights) is correctly implemented and matches the literature pattern from Section 1.2 — but its output is currently ignored for the 'person' class (see Section 4).

4. The Most Important Finding: a Bug That Undermines the Hackathon Goal

In backend/decision/engine.py, inside DecisionEngine.process(), the logic branches on class name:

```
if class_name == "person": is_critical = True # ... immediately escalates to ALARM_ACTIVE, subject only to a cooldown timer – no confidence check, no temporal confirmation, no fusion score check
```

Compare this to the project's own master specification (01_MASTER_PROJECT_SPECIFICATION.pdf, Section 7), which explicitly says: high-priority immediate escalation should require **"person confidence >= configurable high threshold (default 0.85) AND inside the restricted polygon"**, and that **"lower-confidence detections use temporal confirmation plus optional radar/ground evidence."**

The implemented code does not check confidence at all for the 'person' class before escalating. Any person detection inside a zone — even at YOLO's minimum configured confidence of 0.50, a single blurry or partially-occluded frame, or a misclassification — immediately fires ALARM_ACTIVE, creates an event, triggers the ESP32 buzzer, and starts evidence capture. The carefully-designed fusion engine (vision + radar + ground + temporal, weighted, with a 0.65 confirmed threshold) is computed but its result is **never consulted** for the person class — it only matters for non-person classes, which don't ring the alarm anyway.

Why this matters for the hackathon specifically: the problem statement's entire premise is that outdoor systems over-alarm on ambiguous person-shaped triggers in natural conditions (a person-shaped shadow, distant blurry human figure in tall grass, brief occlusion by foliage). This is precisely the failure mode this bug reproduces — it is the one class where false-alarm resilience matters most, and it's the one class where the resilience logic is currently skipped. A judge who reads the master spec alongside the code (as this report did) would flag this immediately as a spec-vs-implementation mismatch.

The fix is small and safe (does not require restructuring): gate the immediate-alarm path behind the configured high-confidence threshold, and route everything below it through the existing temporal/fusion path that already exists in the same file for other classes. This is the #1 item in the Antigravity prompt in Section 10 / the companion .md file.

5. File-by-File Map and How to Run

5.1 Backend (Python / FastAPI)

File / folder	What it does
backend/app.py	Main FastAPI app. Owns the lifespan startup (Supabase → camera → YOLO → ESP32), the async processing loop that ties detection → zone → fusion → decision → event → buzzer → broadcast, and the /ws/stream WebSocket.
backend/config.py	Loads .env into typed constants — camera index, YOLO thresholds, fusion weights, timing constants. Single source of truth for tunable values.
backend/camera/capture.py	Threaded OpenCV camera capture, decoupled from inference so a slow YOLO pass never blocks the live video feed.
backend/camera/health_monitor.py	Tracks camera image-quality metrics (brightness, blur, etc.) shown on the Camera Health page.
backend/vision/detector.py	YOLO11n inference thread. Filters to a fixed set of COCO classes of interest (person, several animals/birds, several vehicle types), runs ByteTrack for track IDs, pushes results to a bounded queue.
backend/vision/zones.py	Polygon restricted-zone engine — ray-casting point-in-polygon test using each detection's foot/bottom-center point. 13/13 unit tests pass.
backend/vision/annotator.py	Draws bounding boxes, zone polygons, labels and decision-state text onto the frame before it's JPEG-encoded and streamed to the frontend.
backend/decision/fusion.py	Weighted evidence fusion (vision/radar/ground/temporal). Correctly implemented — see Section 4 for the caveat about it being bypassed for 'person'.
backend/decision/engine.py	The state machine (NO_DETECTION → ... → EVENT_RESOLVED) per (track_id, zone_id). Contains the bug described in Section 4.
backend/sensors/provider.py	Abstract SensorProvider interface + SimulatedSensorProvider. Designed so a future real-radar/ground provider can be swapped in without touching the decision engine — good design.
backend/events/manager.py	Creates/dedupes/cool-down events, writes to Supabase, tracks active events.
backend/evidence/capture.py	Rolling pre-event frame buffer + post-event recording + snapshot capture.
backend/evidence/storage.py	Uploads captured snapshot/video to Supabase Storage buckets.
backend/hardware/esp32.py	HTTP REST client to the ESP32 (status, sensors, alarm trigger/stop) plus a heartbeat thread. Designed to fail gracefully — vision keeps running if ESP32 is offline.
backend/database/supabase_client.py	Service-role Supabase client (backend only — the key never reaches the frontend).
backend/api/*.py	REST route handlers: health, events, zones, sensors, devices, esp32, settings — one file per resource, mounted under /api in app.py.
esp32/firmware/borderpulse_esp32.ino	Arduino firmware for the ESP32 — buzzer control + (future) ground/radar GPIO reads + Wi-Fi HTTP server. Written but not yet flashed; Wi-Fi credentials are still placeholders.

File / folder	What it does
tests/test_zones.py, tests/test_decision.py	24 automated unit tests, currently all passing.

5.2 Frontend (React + Vite + Tailwind)

File / folder	What it does
frontend/src/App.jsx	React Router setup for the 10 pages + shared Sidebar/StatusBar layout.
frontend/src/contexts/StreamContext.jsx	Owns the WebSocket connection to /ws/stream with auto-reconnect; every page reads live data from here via useStream().
frontend/src/services/api.js	All REST calls (events, zones, sensors, devices, settings) and the WebSocket URL — the single place that knows the backend's base URL.
frontend/src/components/	Shared UI — Sidebar, StatusBar, CameraFeed (renders the base64 JPEG stream), and a small ui.jsx library (Card, Btn, StatusDot, ConfBar, SectionHeader, etc.) used across all pages.
frontend/src/pages/Overview.jsx	Dashboard home — stat cards, camera feed, system health list, sensor-fusion status, active alerts. Reviewed directly: functional, coherent, correctly reflects live + simulated states.
frontend/src/pages/LiveMonitor.jsx	Full live view — camera feed, FPS/latency/object-count diagnostics, decision-state banner, sensor evidence panel, detection list with confidence bars, and simulation toggles for radar/ground. Reviewed directly: functional.
frontend/src/pages/Zones.jsx	Interactive polygon zone editor drawn on an HTML canvas over the live frame — click to add points, save/enable/disable/delete zones. Reviewed directly: functional and well-built.
frontend/src/pages/Devices.jsx	Camera + ESP32 hardware status, buzzer test button, wiring/setup notes inline. Reviewed directly: functional.
frontend/src/pages/Events.jsx, EventDetail.jsx	Event log list and single-event detail (evidence, fusion breakdown, ack/resolve/false-positive actions). Not directly opened in this pass — recommend a follow-up review since the user specifically flagged 'some pages not fully corrected.'
frontend/src/pages/Sensors.jsx, Settings.jsx, CameraHealth.jsx, Analytics.jsx	Sensor state/simulation controls, fusion-weight/decision-config sliders, camera quality metrics, event statistics. Not directly opened in this pass — same recommendation as above.

5.3 How to run it

Terminal 1 — backend:

```
cd d:\hackelite
python -m uvicorn backend.app:app --host 0.0.0.0 --port 8000
```

Terminal 2 — frontend:

```
cd d:\hackelite\frontend
npm run dev
```

Then open <http://localhost:5173/>. Run tests with `python -m pytest tests/ -v` (expect 24/24 passed). The `.env` file already has a real Supabase project connected (`frmisnduadstnjwyyvym`) — do not recreate it.

6. What's Broken, Unfinished, or Only Simulated Right Now

Item	Status	Action needed
Person-class alarm bypass	Bug (Section 4)	Gate immediate escalation behind YOLO_HUMAN_HIGH_CONFIDENCE (0.85); route lower-confidence person detections through the existing temporal/fusion path.
ESP32 firmware	Written, not flashed	Confirm exact board variant, set real Wi-Fi credentials in the .ino file, flash via Arduino IDE, update ESP32_IP in .env.
Radar sensor	Simulated only; hardcoded 'RADAR — NOT CONNECTED' in the live stream payload in app.py regardless of the simulation toggle	Either wire a real RCWL-0516 module, or at minimum make the simulated-radar toggle actually reflect in the stream payload (currently it doesn't, which is itself a small bug — the toggle changes fusion input but the UI always shows OFFLINE).
Ground sensor	Simulated by default; real-ESP32 path exists in code	Same as radar — needs physical wiring to move from SIMULATED to REAL.
Evidence capture end-to-end	Implemented, not verified live	Draw a zone, trigger a real intrusion, confirm a .jpg and .mp4 actually land in evidence_local/ and get uploaded to Supabase Storage.
Git repository	None exists yet (per HANDOFF_FOR_NEXT_AI.md)	Initialize git and commit — a hackathon submission with no version history looks unfinished to judges, and is also just good practice at this stage.
requirements.txt version conflict	Already fixed	fastapi and websockets pins were corrected on 2026-08-23; no action needed, just don't revert it.
Supabase service-role key	Present in .env, needs a final human check	Confirm it's the real key (not a placeholder) before final demo; confirm it is never referenced anywhere under frontend/.
Events / EventDetail / Sensors / Settings / Analytics / CameraHealth pages	Not directly inspected in this pass	You mentioned some pages 'aren't fully corrected' — worth a dedicated follow-up read of these six files specifically, since the four pages this report did open (Overview, LiveMonitor, Zones, Devices) were all solid.

7. UI/UX Assessment

The four pages reviewed directly (Overview, Live Monitor, Zones, Devices) use a consistent dark "security-ops" theme — dark navy background, cyan accent, monospace stat labels, status dots, a subtle grid background. This is a reasonable and appropriately serious visual direction for a surveillance dashboard, and it is not a generic AI-template look — it reads more like a real operations console. That's a good foundation to build on rather than replace.

Where it can look more polished, without touching page structure or logic:

- Replace plain unicode glyphs used as icons (■ ■ ■ ■ ✓ ✕) with a real icon set (e.g. lucide-react) for crisper, consistent-weight icons at every size — unicode glyphs render inconsistently across fonts/OSes and are one of the more obvious 'not-quite-finished' tells.
- Standardize spacing and card padding across pages — some cards use tight py-1.5 rows, others more generous spacing; a single spacing scale (e.g. Tailwind's default 2/3/4/6 steps used consistently) reads as more deliberate.
- Add a consistent empty/loading state pattern (skeletons instead of plain spinners) for a more finished feel on first load.
- For the six pages not reviewed in this pass, do a visual QA pass specifically for: broken layouts on smaller screens, any placeholder/lorem text, and any console errors on load — since these are the pages you flagged as not fully corrected.
- Consider a single typographic accent (e.g. a slightly heavier weight for state labels like ALARM_ACTIVE / CRITICAL) so the eye is drawn to the most important information first on the Live Monitor and Overview pages during a live demo.

None of this requires restructuring the page list, routes, or component tree — it's a styling/detail pass on top of what's already there.

8. New Feature Recommendations (Prioritized)

8.1 Must-fix before demo

- **Fix the person-class bypass (Section 4)** — this is the single highest-value fix relative to the problem statement's wording, and it's a small, contained code change.
- **Get the ESP32 buzzer path physically working end-to-end** — a hackathon demo of a 'hardware system' needs the hardware to actually respond at least once live.
- **Add a visible False-Alarm-Rate / Precision metric to the Analytics page** — right now the system logs events but there's no dashboard number that directly answers the judges' likely first question: "how do you know this actually reduces false alarms?" Compute it from the existing false_positive vs. confirmed event resolution field already stored per event.

8.2 High-value additions that directly answer the problem statement

- **Wind-adaptive confirmation threshold:** pull a wind-speed reading (a cheap anemometer on the ESP32, or a local weather API as a stand-in for the demo) and raise TEMPORAL_MIN_FRAMES / lower the fusion weight given to a single vision frame when wind speed crosses a set point. This is exactly the pattern used by the commercial systems cited in Section 1.2, and it directly answers 'wind' in the problem statement, which nothing in the current build addresses explicitly.
- **Bounding-box jitter/stability filter:** require a track's bounding box to be reasonably stable (size/position not flickering wildly frame-to-frame) before counting a frame toward temporal confirmation — foliage-triggered false detections tend to be unstable/flickering compared to a walking person.
- **Expose the fusion score breakdown in the Event Detail page:** the FusionResult.evidence dict already contains vision/radar/ground/temporal contributions — surfacing this per event as a simple bar breakdown gives an operator (and a judge) a transparent 'why did this alarm fire' explanation, which is a strong selling point for a false-alarm-resilience pitch.
- **Real radar integration (RCWL-0516):** currently weighted at 0.20 in fusion but hardcoded offline in the UI — wiring even a basic radar module and getting it reporting real triggers would materially strengthen the 'multi-sensor fusion' story for judges.

8.3 Good-to-have polish

- SMS/Telegram/email escalation for CRITICAL events (mirrors the CIBMS command-center hand-off pattern from Section 1.1) — even a simple webhook to a Telegram bot is enough for a demo.
- A false-positive labeling workflow in the UI (already partially there via 'false positive' resolution) feeding into a simple stats view, framed explicitly as the first step toward eventually calibrating thresholds from real data rather than fixed defaults.
- Night/low-light note in documentation: the current RGB webcam has no IR/thermal capability — flag this honestly to judges as a known limitation and future-hardware item rather than leaving it unaddressed.

9. YOLO Model, Detection Classes, and Sensor Thresholds — Reference

9.1 What the YOLO model actually detects

The system uses **YOLO11n** (Ultralytics), a pretrained model on the COCO dataset, restricted in code (backend/vision/detector.py, CLASSES_OF_INTEREST) to a fixed subset relevant to intrusion monitoring:

- **Human:** person
- **Wildlife/animals:** bird, cat, dog, horse, sheep, cow, elephant, bear, zebra, giraffe
- **Vehicles:** car, motorcycle, bus, truck, bicycle

Important clarification for the demo pitch: pretrained COCO/YOLO does *not* detect weather phenomena like rain, storms, or 'natural disasters' as objects — YOLO detects discrete physical objects in a frame, not atmospheric conditions. Trying to make YOLO 'detect rain' is the wrong tool for that job. The correct approach (and what the project's own master spec implies, Section 6/9) is a **separate environmental input** — a cheap rain/moisture sensor on the ESP32, or a weather API call — that adjusts detection thresholds or suppresses the ground-sensor channel during heavy rain, rather than expecting the vision model to classify weather. Similarly, YOLO alone does not prove 'fighting', 'military activity' or generic 'suspicious behaviour' — the project's spec correctly flags this and instead recommends measurable secondary analytics (person count, proximity, speed, dwell time, repeated boundary crossing) as a future behaviour layer, not a claim about the base object detector.

9.2 Current threshold and weight values

Parameter	Default value	.env variable
YOLO base confidence	0.50	YOLO_CONFIDENCE
Human high-confidence fast-path	0.85	YOLO_HUMAN_HIGH_CONFIDENCE
YOLO inference size	640 px	YOLO_IMGSZ
Vision fusion weight	0.55	FUSION_WEIGHT_VISION
Radar fusion weight	0.20	FUSION_WEIGHT_RADAR
Ground fusion weight	0.15	FUSION_WEIGHT_GROUND
Temporal fusion weight	0.10	FUSION_WEIGHT_TEMPORAL
Fusion confirmed threshold	0.65 (of 1.0)	FUSION_CONFIRMED_THRESHOLD
Minimum frames for temporal confirmation	3 frames	TEMPORAL_MIN_FRAMES
Temporal confirmation window	1.0 second	TEMPORAL_WINDOW_SECONDS
Event cooldown (dedup)	10 seconds	EVENT_COOLDOWN_SECONDS
Pre-event evidence buffer	5 seconds	PRE_EVENT_SECONDS
Post-event evidence recording	8 seconds	POST_EVENT_SECONDS

9.3 Radar and ground sensor — what they actually mean

- **Radar (RCWL-0516 planned):** presence/motion evidence only. It cannot identify what triggered it, give exact coordinates, or confirm a human specifically — the project's own spec is explicit about this. It should only ever contribute a supporting weight in fusion, never fire an alarm alone.
- **Ground/vibration sensor:** physical disturbance evidence (a footstep, digging, a vehicle) — also not human-specific on its own. Code correctly labels this 'GROUND_TRIGGER', not 'HUMAN_DETECTED'.
- **The decision-engine state machine** (backend/decision/engine.py) has ten states: NO_DETECTION → POSSIBLE_DETECTION → TEMPORAL_CONFIRMATION → CONFIRMED → ALARM_ACTIVE → EVIDENCE_CAPTURE → EVENT_ACTIVE → EVENT_RESOLVED, with FALSE_POSITIVE and ACKNOWLEDGED as terminal/override states. This is a solid, correctly-designed state machine — the only issue is that the 'person' class currently skips straight from detection to ALARM_ACTIVE instead of passing through the temporal/confirmation states, per Section 4.

10. The Antigravity Fix Prompt

A ready-to-paste prompt for Antigravity (or any coding agent continuing this project) has been written to a companion file: `ANTIGRAVITY_FALSE_ALARM_FIX_PROMPT.md`. It is structured so the agent fixes issues in priority order without re-architecting anything, and includes explicit 'do NOT' guardrails (matching the project's own `HANDOFF_FOR_NEXT_AI.md` conventions) so it doesn't regenerate the project from scratch or touch files marked as already-working.

The prompt covers, in order:

- 1. The person-class alarm bypass fix (Section 4) — the highest-priority item
- 2. Making the radar simulation toggle actually reflect in the live stream payload
- 3. A false-alarm-rate metric on the Analytics page
- 4. Exposing the fusion score breakdown on the Event Detail page
- 5. A wind-adaptive confirmation threshold (stub-able with a manual slider if no real anemometer is available for the demo)
- 6. UI polish pass (icons, spacing) without structural changes
- 7. ESP32 physical bring-up checklist (confirm board, flash, verify buzzer)
- 8. Initializing git version control for the project

Give the agent this report and the companion prompt file together — the report gives it the *why* behind each fix (so it doesn't 'fix' things in a way that breaks the false-alarm-resilience story), and the prompt file gives it the concrete *what* to do.

End of report.